Lab 08

Data Manipulation

Objective

The objective of this lab is to learn how to create, query, update, and drop views in MySQL.

Introduction

A view is a virtual table based on the result of a SELECT query. It does not store data itself; it shows data from one or more base tables every time it is queried. Views are used to simplify complex queries, to hide sensitive columns (such as salary or email) from users, and to give different users different perspectives of the same data. In this lab, two tables (Departments and Employees) are created in a database called university_db, and different types of views are built on them: simple views, filtered views, join views, and aggregate views. The views are then queried, updated through, replaced, and finally dropped.

Procedure

1. Start WampServer and open phpMyAdmin in the browser.

2. Open the SQL tab and create the database university_db, then select it.

3. Create the Departments table and insert its four records.

4. Create the Employees table (with the foreign key to Departments) and insert its eight records.

5. Create Employee_Contact_View as a simple view of Emp_ID, Emp_Name and Email.

6. Create High_Salary_View with a WHERE condition, then query it with ORDER BY Salary DESC.

7. Create Employee_Department_View using INNER JOIN, then query it for Computer Science.

8. Create Department_Summary_View using COUNT, AVG, SUM and GROUP BY, then join it with Departments.

9. Create Employee_Salary_View, update a salary through it, and check the result in both the view and the Employees table.

10. Change High_Salary_View with CREATE OR REPLACE VIEW and query it again.

11. Drop the views using DROP VIEW and DROP VIEW IF EXISTS.

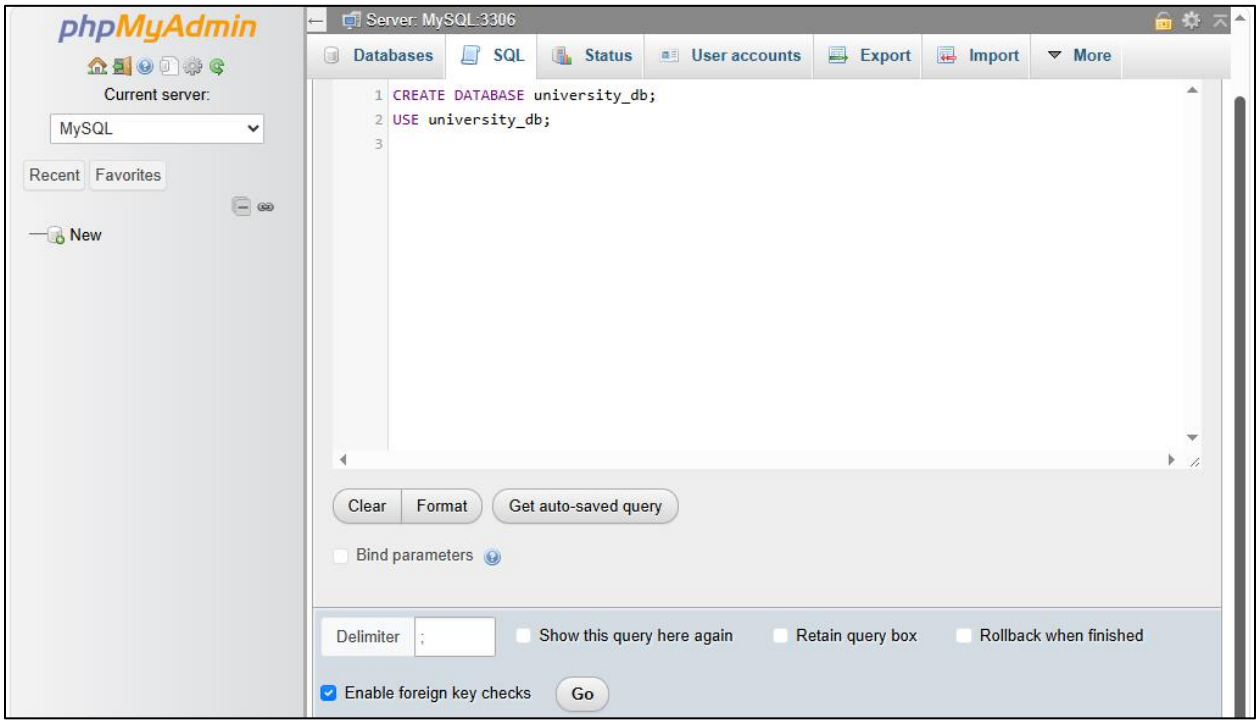
12. Take a screenshot of each query output.

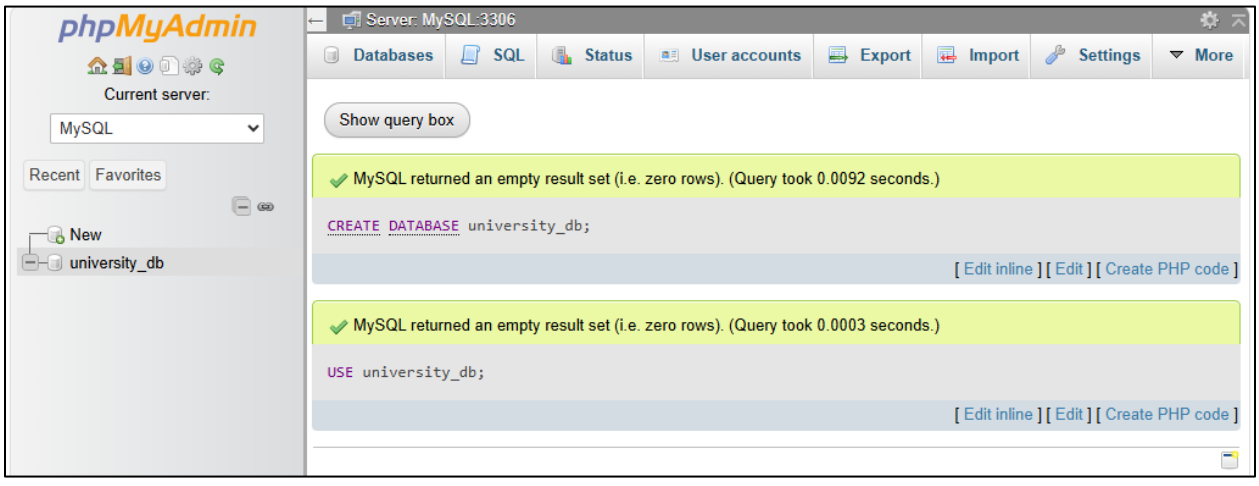
Lab Task

Task 1: Create the database

CREATE DATABASE university_db;

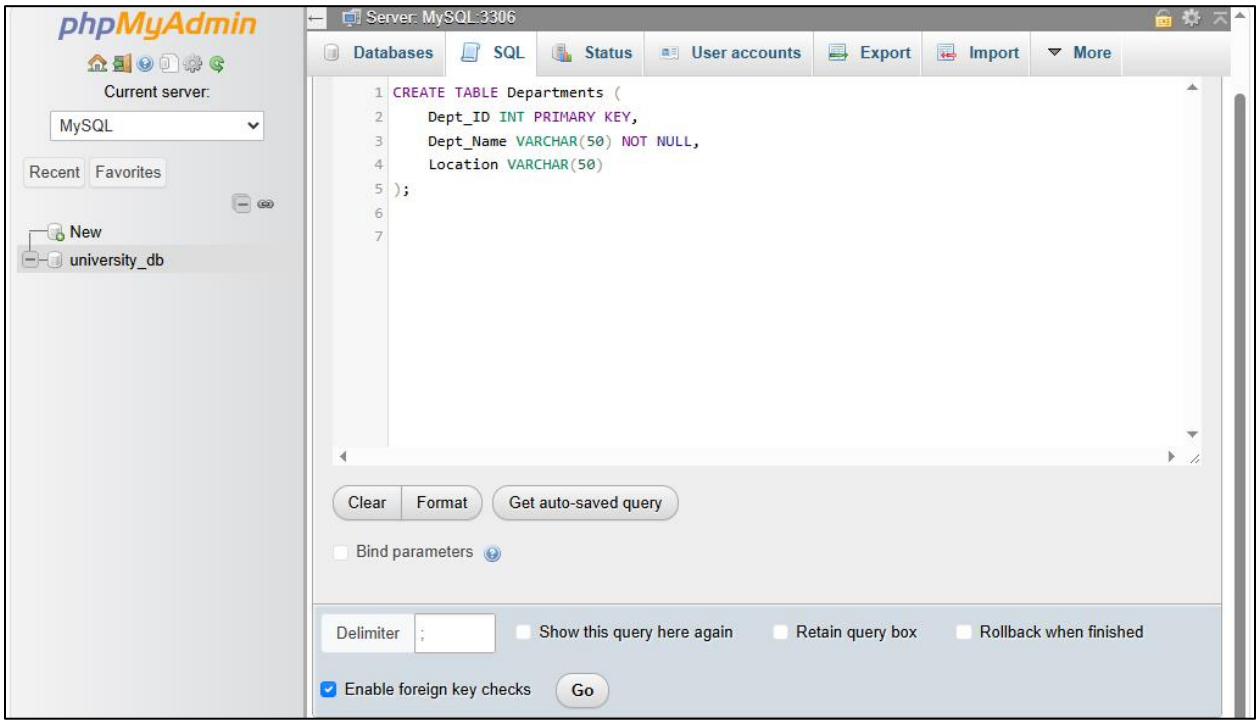
USE university_db;

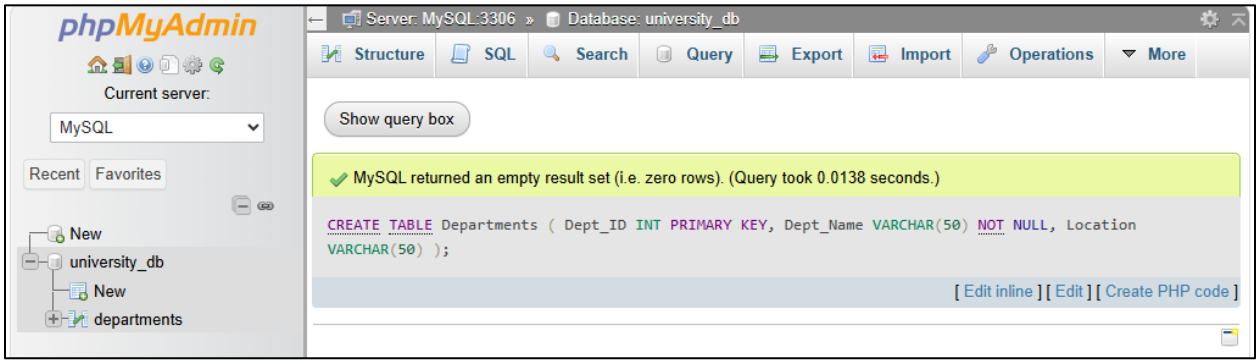




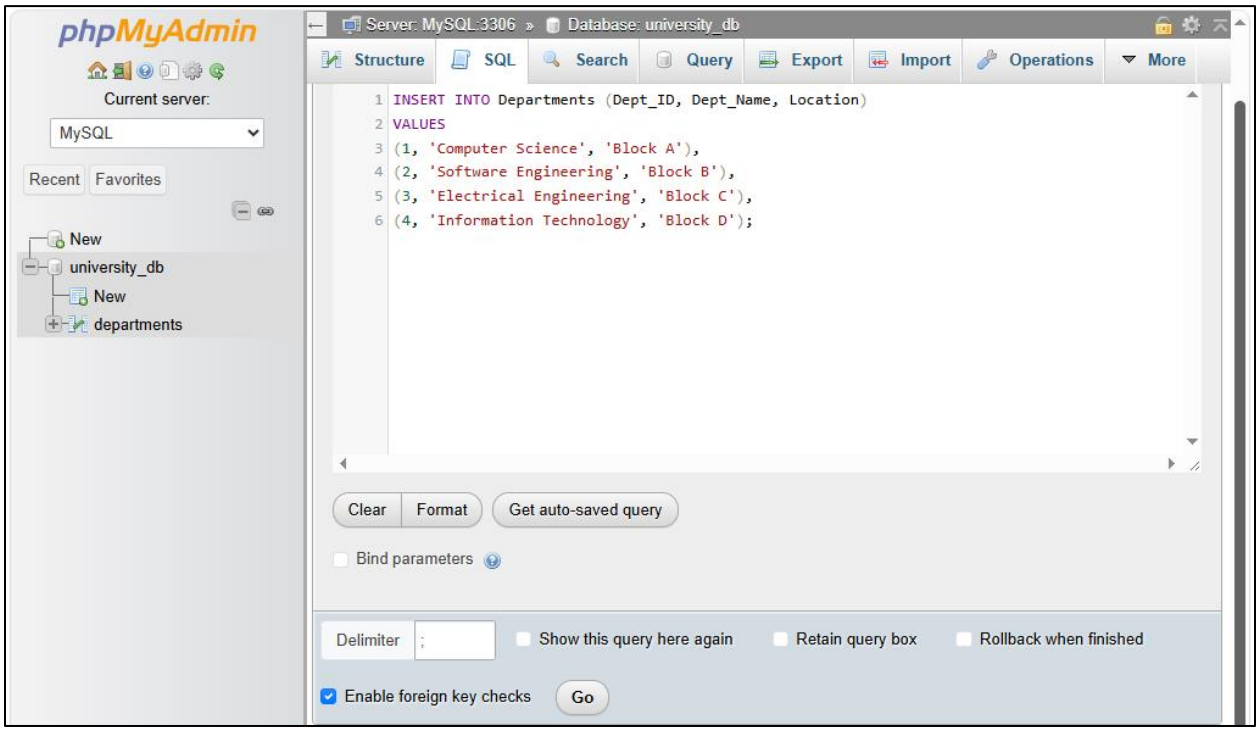
Task 2: Create the Departments table and insert data

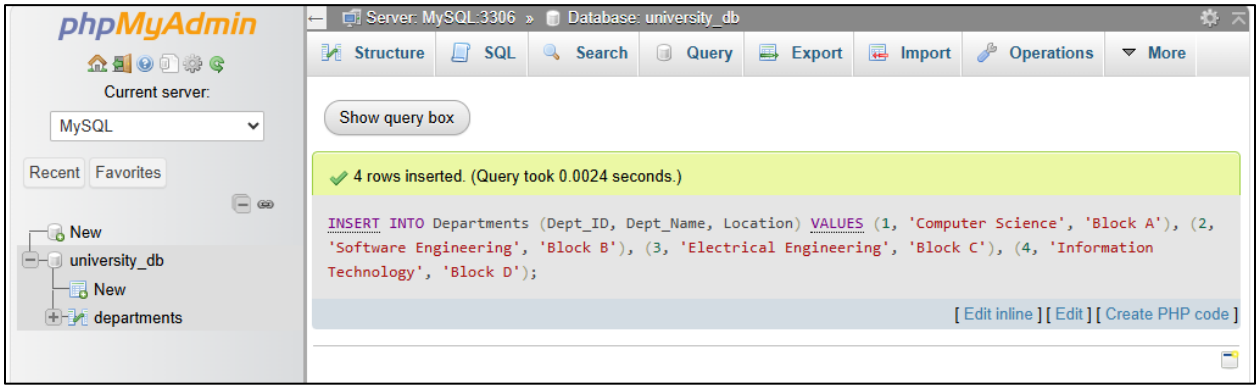
```
CREATE TABLE Departments (
   Dept_ID INT PRIMARY KEY,
   Dept_Name VARCHAR(50) NOT NULL,
   Location VARCHAR(50)
);
```





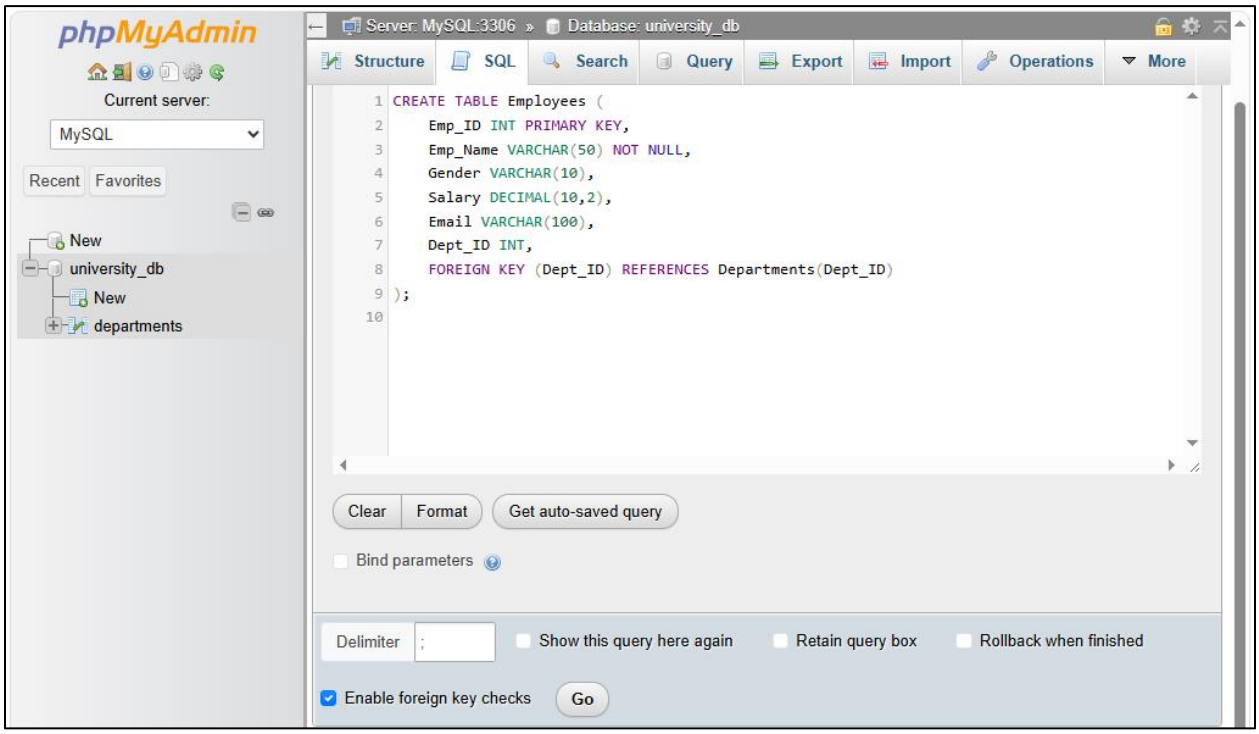

```
INSERT INTO Departments (Dept_ID, Dept_Name, Location)
VALUES
(1, 'Computer Science', 'Block A'),
(2, 'Software Engineering', 'Block B'),
(3, 'Electrical Engineering', 'Block C'),
(4, 'Information Technology', 'Block D');
```

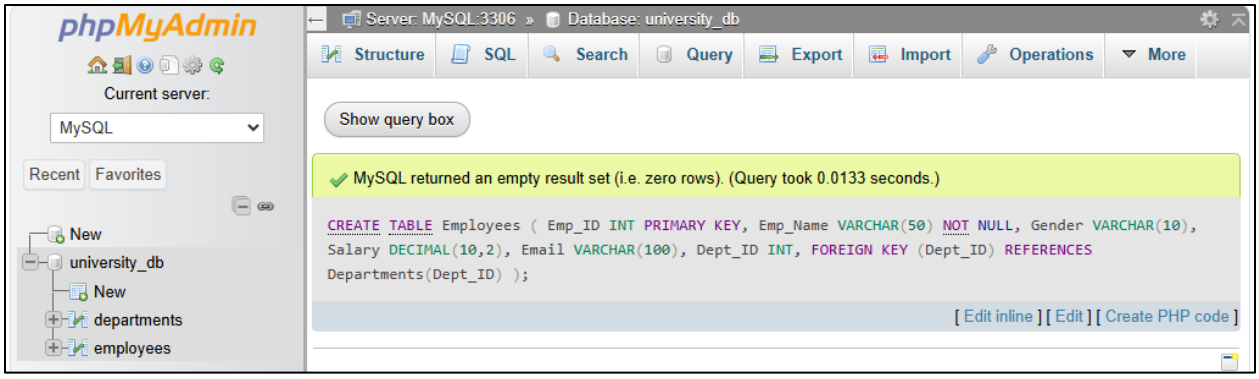




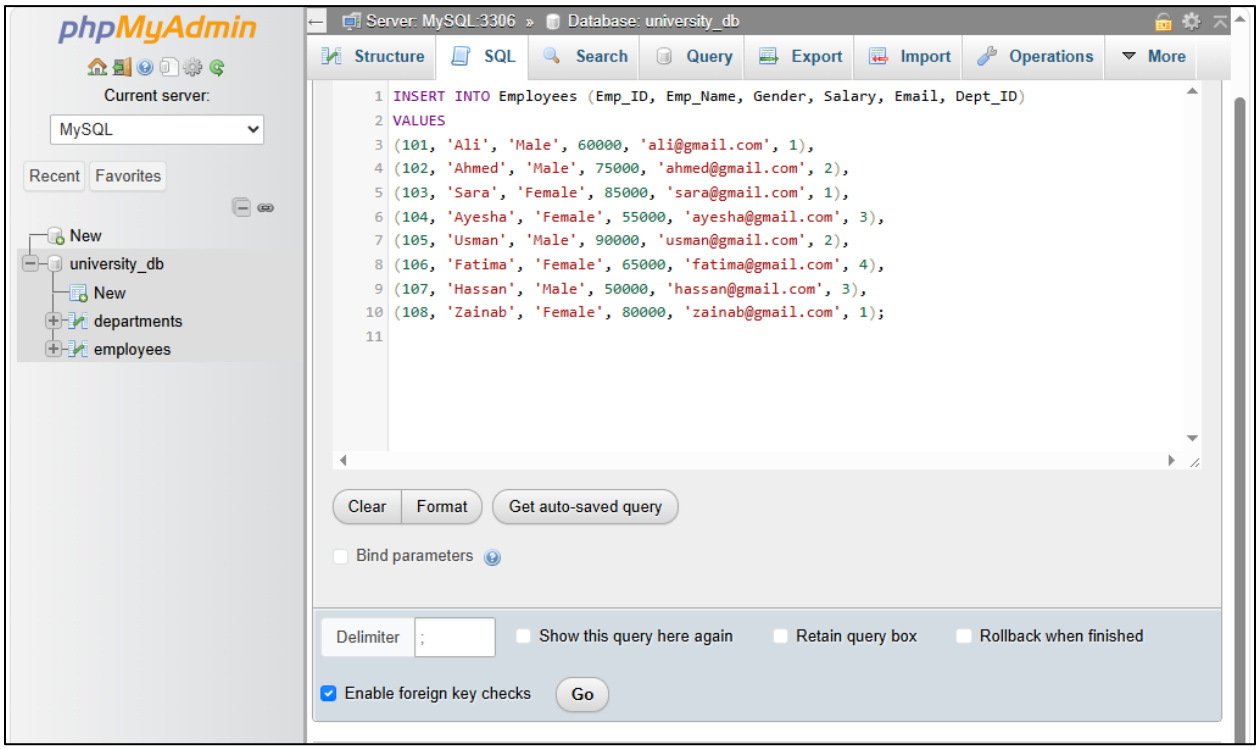
Task 3: Create the Employees table and insert data

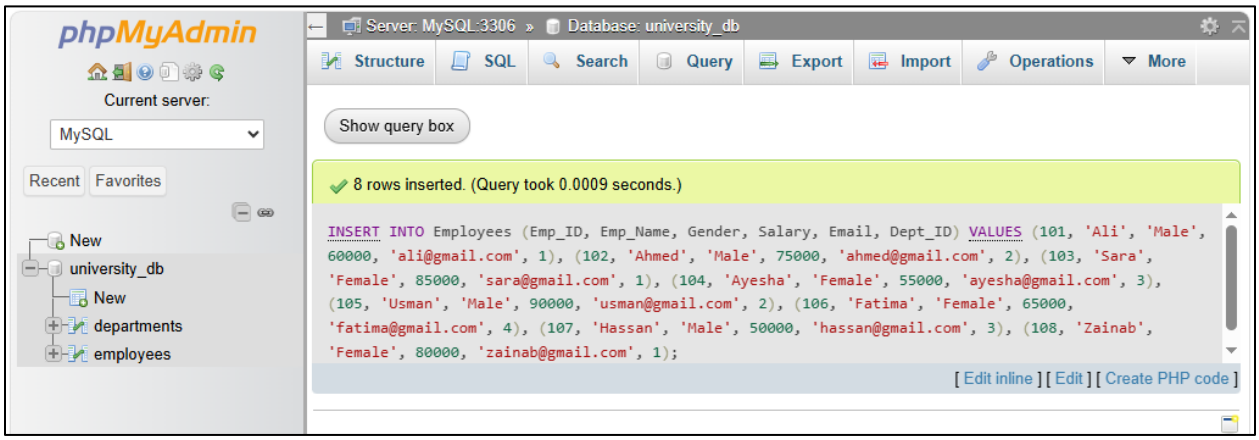
```
CREATE TABLE Employees (
   Emp_ID INT PRIMARY KEY,
   Emp_Name VARCHAR(50) NOT NULL,
   Gender VARCHAR(10),
   Salary DECIMAL(10,2),
   Email VARCHAR(100),
   Dept_ID INT,
   FOREIGN KEY (Dept_ID) REFERENCES Departments(Dept_ID)
);
```





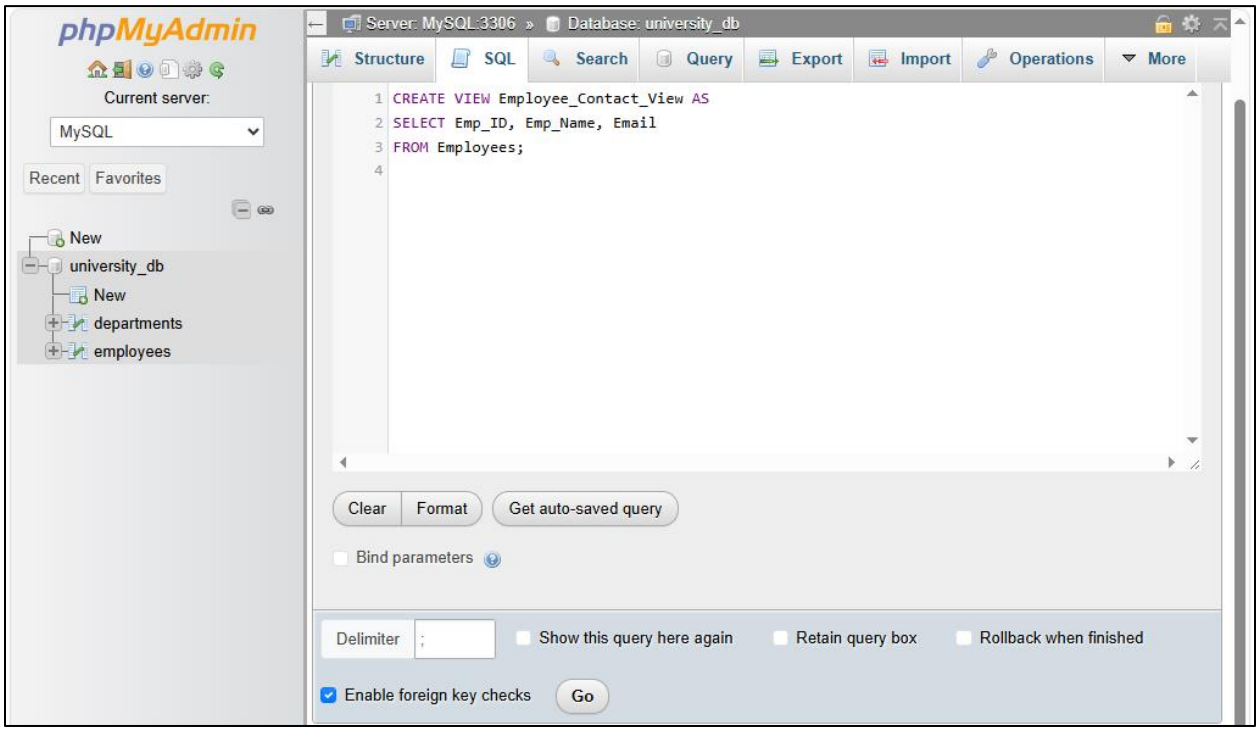
INSERT INTO Employees (Emp_ID, Emp_Name, Gender, Salary, Email, Dept_ID)
VALUES
(101, 'Ali', 'Male', 60000, 'ali@gmail.com', 1),
(102, 'Ahmed', 'Male', 75000, 'ahmed@gmail.com', 2),
(103, 'Sara', 'Female', 85000, 'sara@gmail.com', 1),
(104, 'Ayesha', 'Female', 55000, 'ayesha@gmail.com', 3),
(105, 'Usman', 'Male', 90000, 'usman@gmail.com', 2),
(106, 'Fatima', 'Female', 65000, 'fatima@gmail.com', 4),
(107, 'Hassan', 'Male', 50000, 'hassan@gmail.com', 3),
(108, 'Zainab', 'Female', 80000, 'zainab@gmail.com', 1);

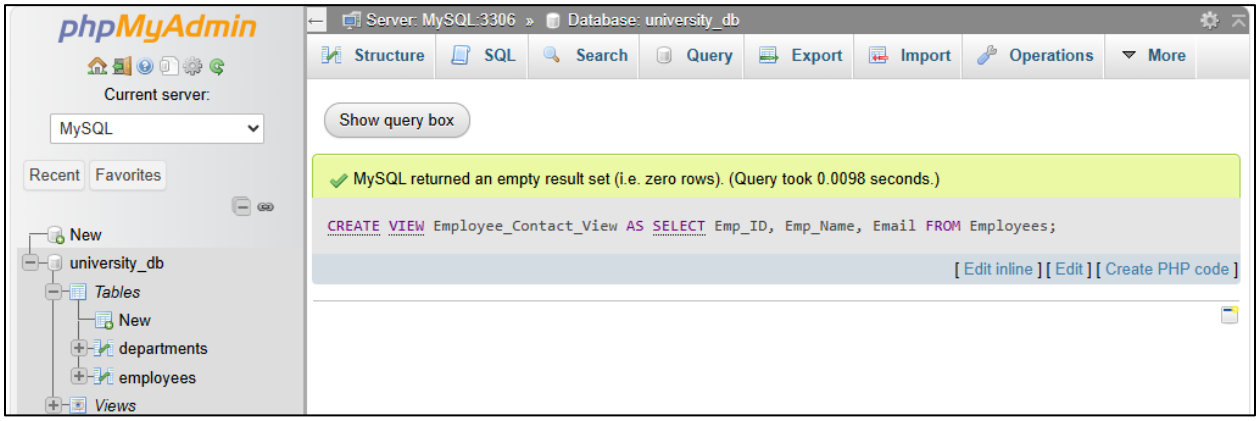




Task 4: Create a simple view

CREATE VIEW Employee_Contact_View AS
SELECT Emp_ID, Emp_Name, Email
FROM Employees;

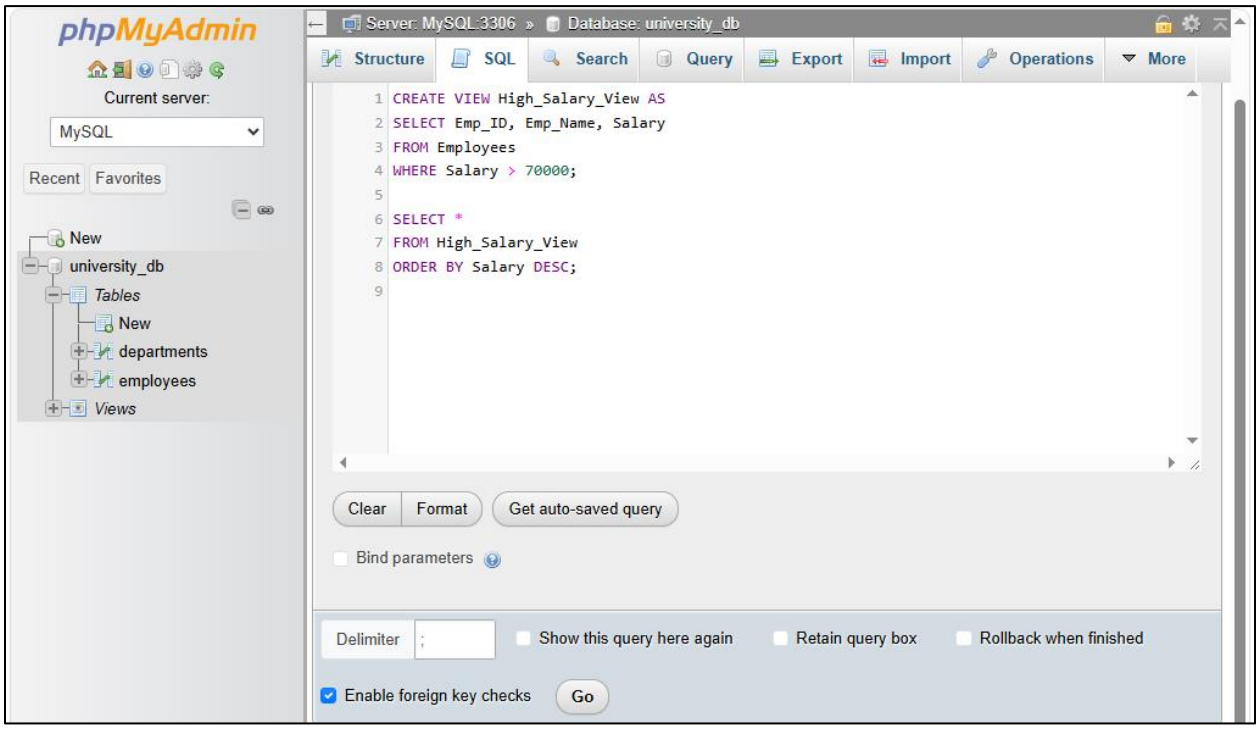


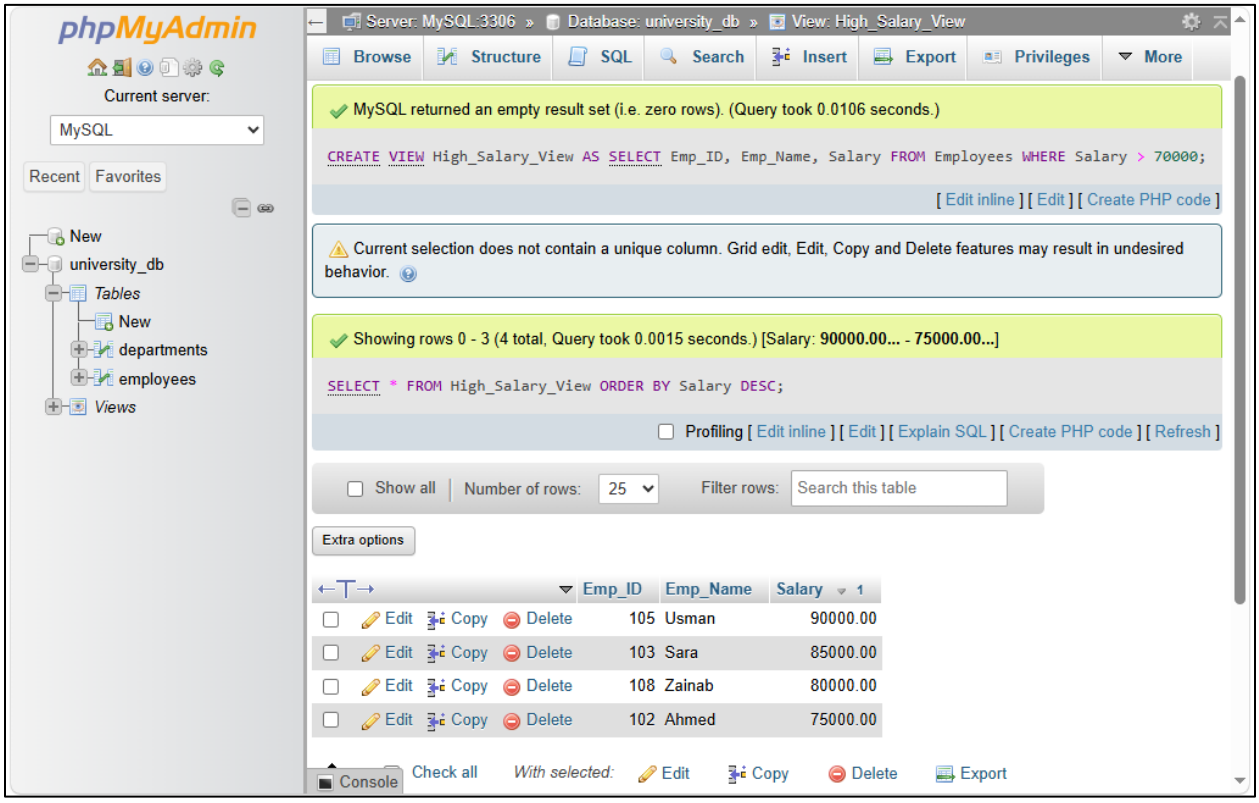


Task 5: Create a view with a condition and query it

```
CREATE VIEW High_Salary_View AS
SELECT Emp_ID, Emp_Name, Salary
FROM Employees
WHERE Salary > 70000;

SELECT *
FROM High_Salary_View
ORDER BY Salary DESC;
```

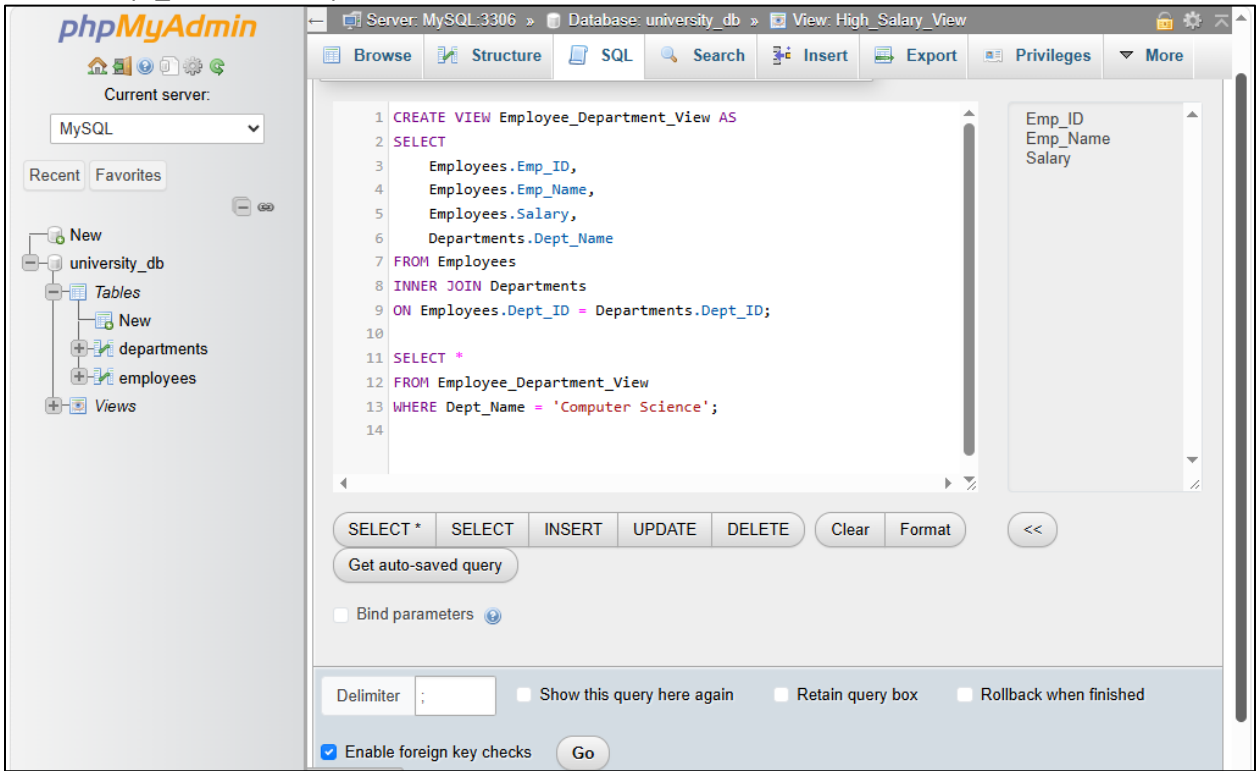

phpMyAdmin
Current server:
MySQL
Recent
Favorites
New
university_db
Tables
New
departments
employees
Views
Server: MySQL:3306 » Database: university_db
Structure
SQL
Search
Query
Export
Import
Operations
More
1 CREATE VIEW High_Salary_View AS
2 SELECT Emp_ID, Emp_Name, Salary
3 FROM Employees
4 WHERE Salary > 70000;
5
6 SELECT *
7 FROM High_Salary_View
8 ORDER BY Salary DESC;
9
Clear
Format
Get auto-saved query
Bind parameters
Delimiter
Show this query here again
Retain query box
Rollback when finished
Enable foreign key checks
Go

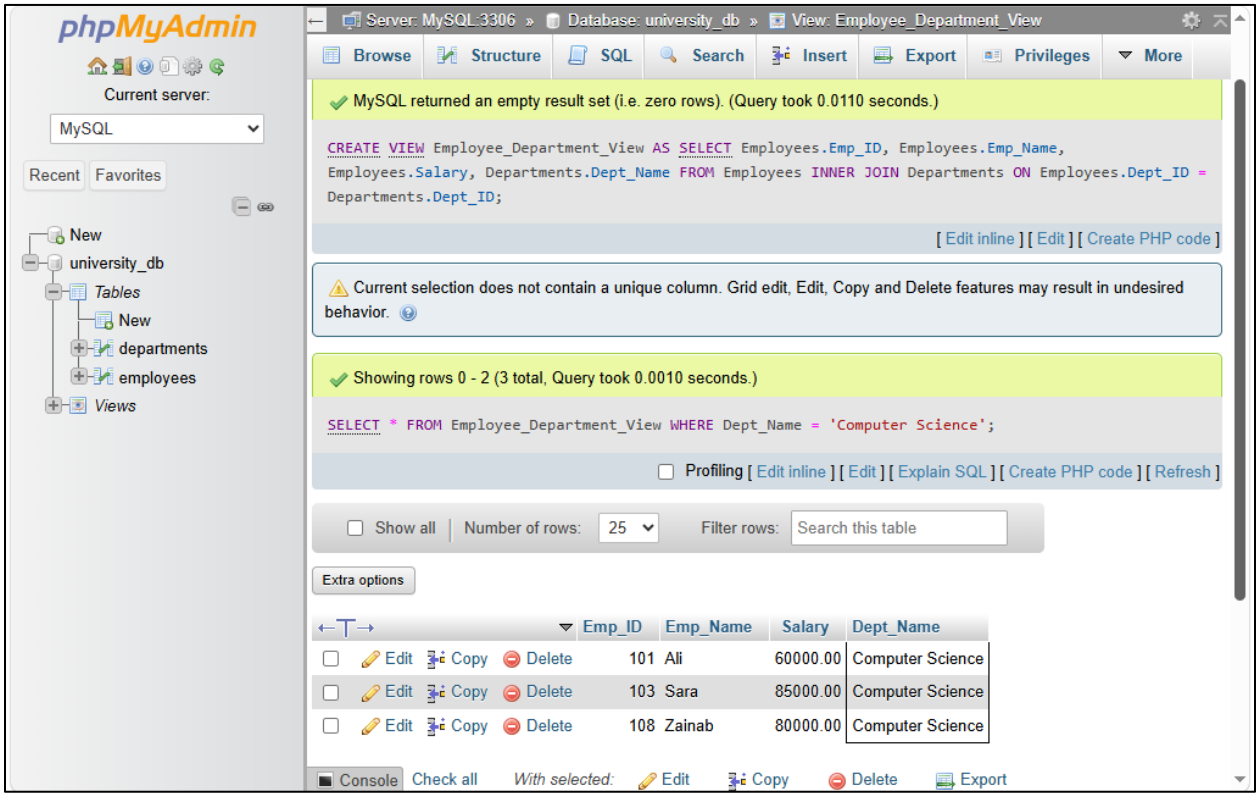

phpMyAdmin
Current server:
MySQL
Recent
Favorites
New
university_db
Tables
New
departments
employees
Views
Server: MySQL:3306 » Database: university_db » View: High_Salary_View
Browse
Structure
SQL
Search
Insert
Export
Privileges
More
MySQL returned an empty result set (i.e. zero rows). (Query took 0.0106 seconds.)
CREATE VIEW High_Salary_View AS SELECT Emp_ID, Emp_Name, Salary FROM Employees WHERE Salary > 70000;
[Edit inline] [Edit] [Create PHP code]
Current selection does not contain a unique column. Grid edit, Edit, Copy and Delete features may result in undesired behavior.
Showing rows 0 - 3 (4 total, Query took 0.0015 seconds.) [Salary: 90000.00... - 75000.00...]
SELECT * FROM High_Salary_View ORDER BY Salary DESC;
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
Show all
Number of rows: 25
Filter rows: Search this table
Extra options
Emp_ID Emp_Name Salary
Edit Copy Delete 105 Usman 90000.00
Edit Copy Delete 103 Sara 85000.00
Edit Copy Delete 108 Zainab 80000.00
Edit Copy Delete 102 Ahmed 75000.00
Console
Check all
With selected:
Edit
Copy
Delete
Export

Task 6: Create a join view and query it

```
CREATE VIEW Employee_Department_View AS
SELECT
   Employees.Emp_ID,
   Employees.Emp_Name,
   Employees.Salary,
   Departments.Dept_Name
FROM Employees
INNER JOIN Departments
ON Employees.Dept_ID = Departments.Dept_ID;

SELECT *
FROM Employee_Department_View
WHERE Dept_Name = 'Computer Science';
```



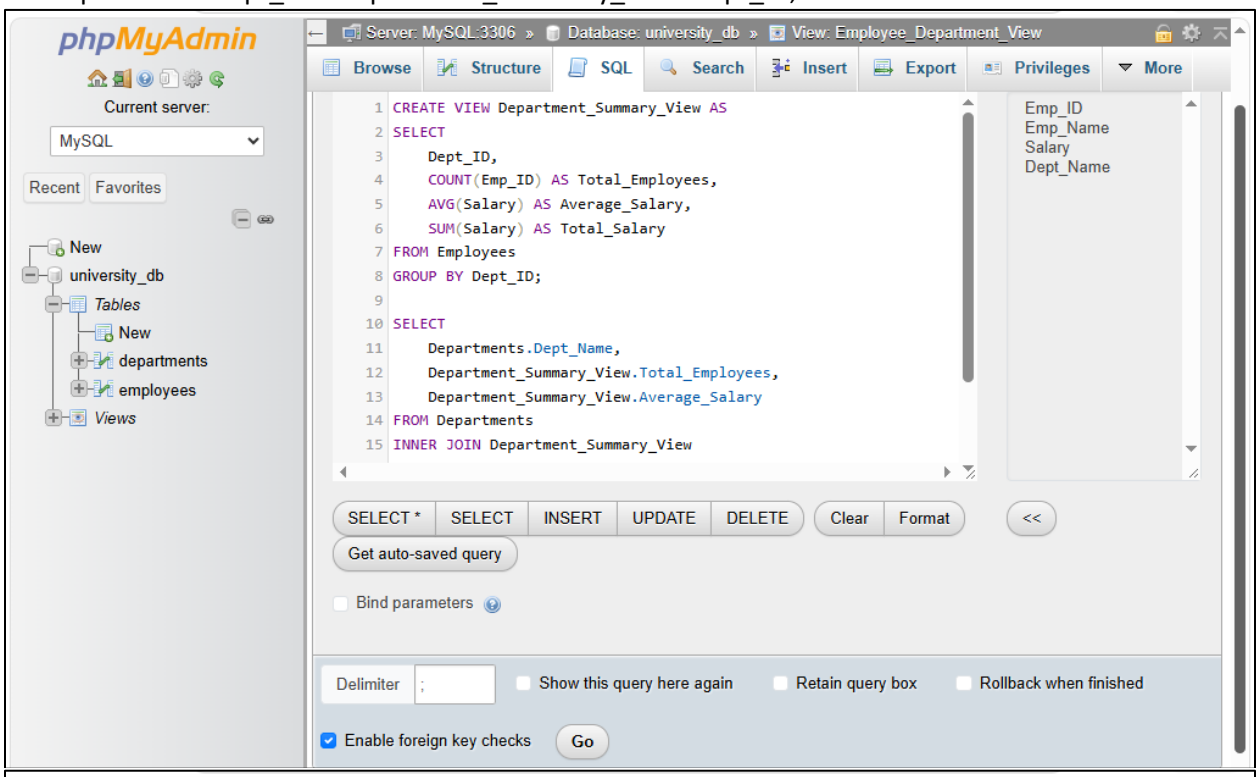


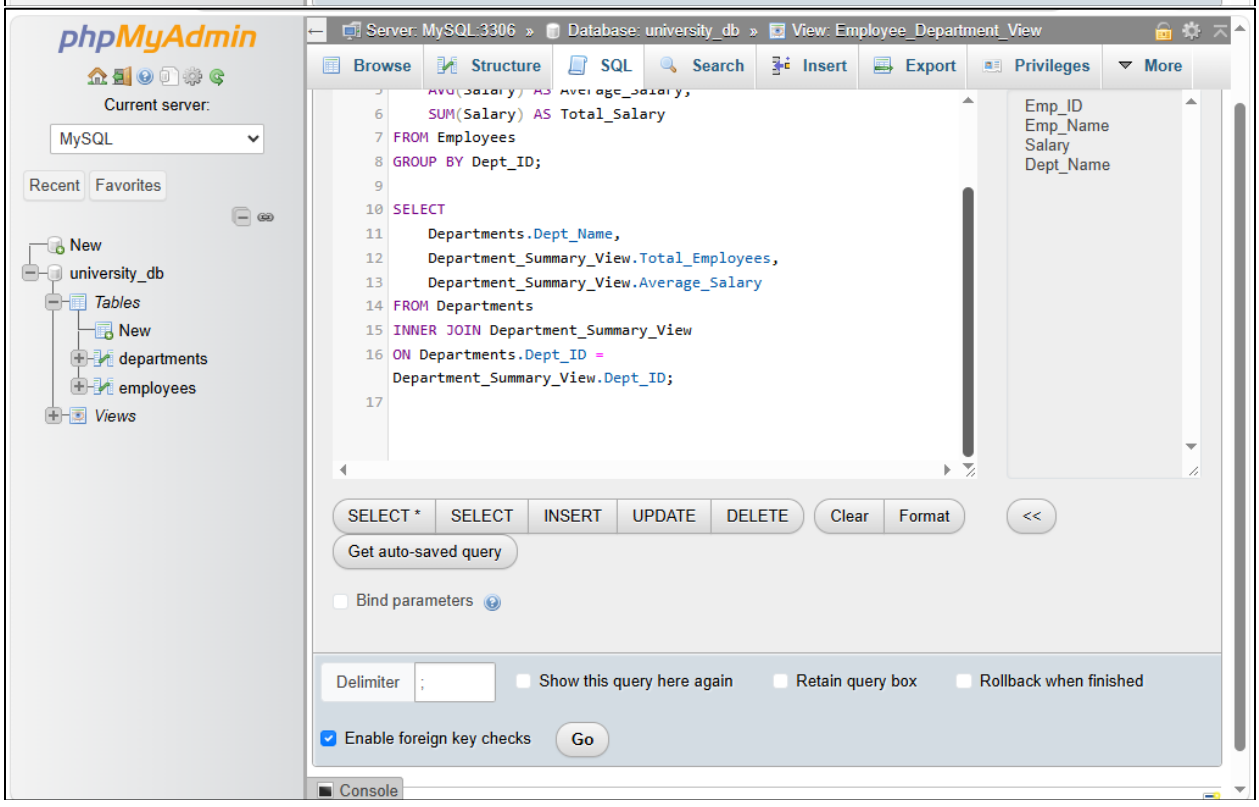
Task 7: Create an aggregate view and join it with a table

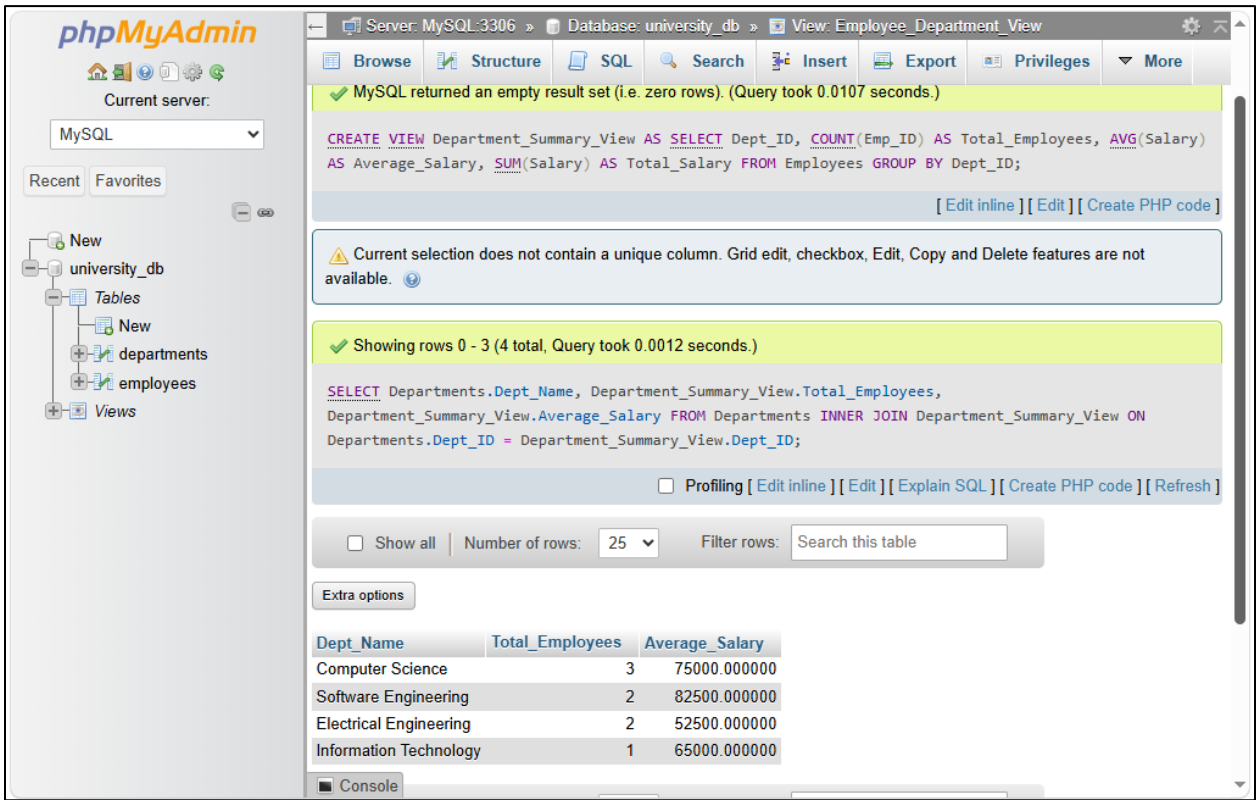
```
CREATE VIEW Department_Summary_View AS
SELECT
   Dept_ID,
   COUNT(Emp_ID) AS Total_Employees,
   AVG(Salary) AS Average_Salary,
   SUM(Salary) AS Total_Salary
FROM Employees
GROUP BY Dept_ID;

SELECT
   Departments.Dept_Name,
   Department_Summary_View.Total_Employees,
   Department_Summary_View.Average_Salary
FROM Departments
INNER JOIN Department_Summary_View
```

ON Departments.Dept_ID = Department_Summary_View.Dept_ID;







Task 8: Update data through a view

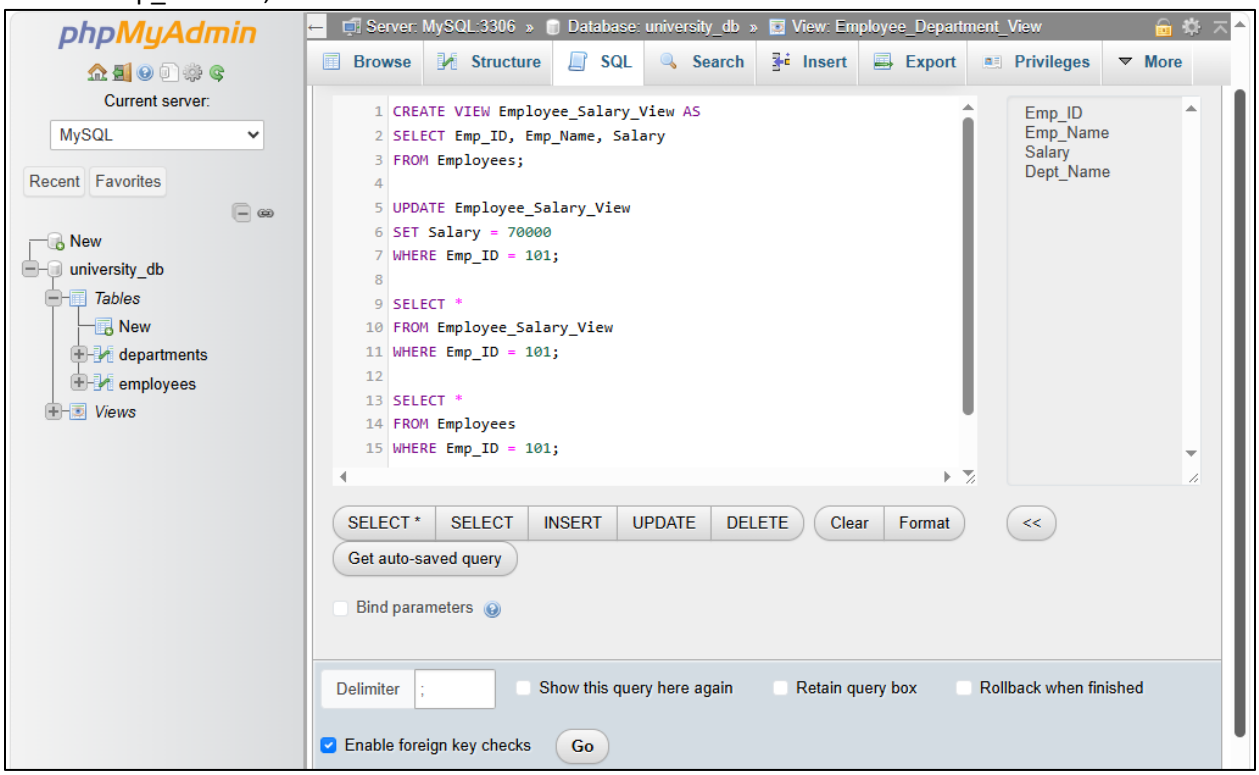
```
CREATE VIEW Employee_Salary_View AS
SELECT Emp_ID, Emp_Name, Salary
FROM Employees;

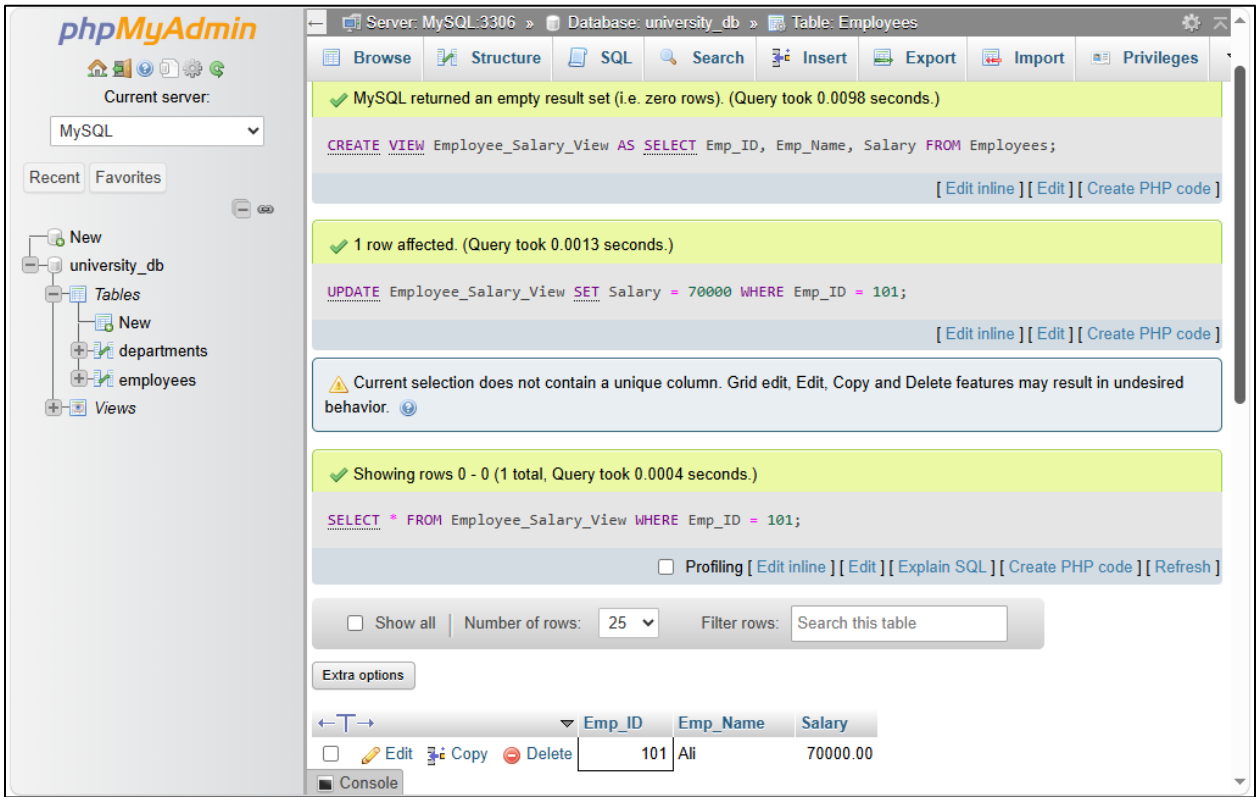
UPDATE Employee_Salary_View
SET Salary = 70000
WHERE Emp_ID = 101;

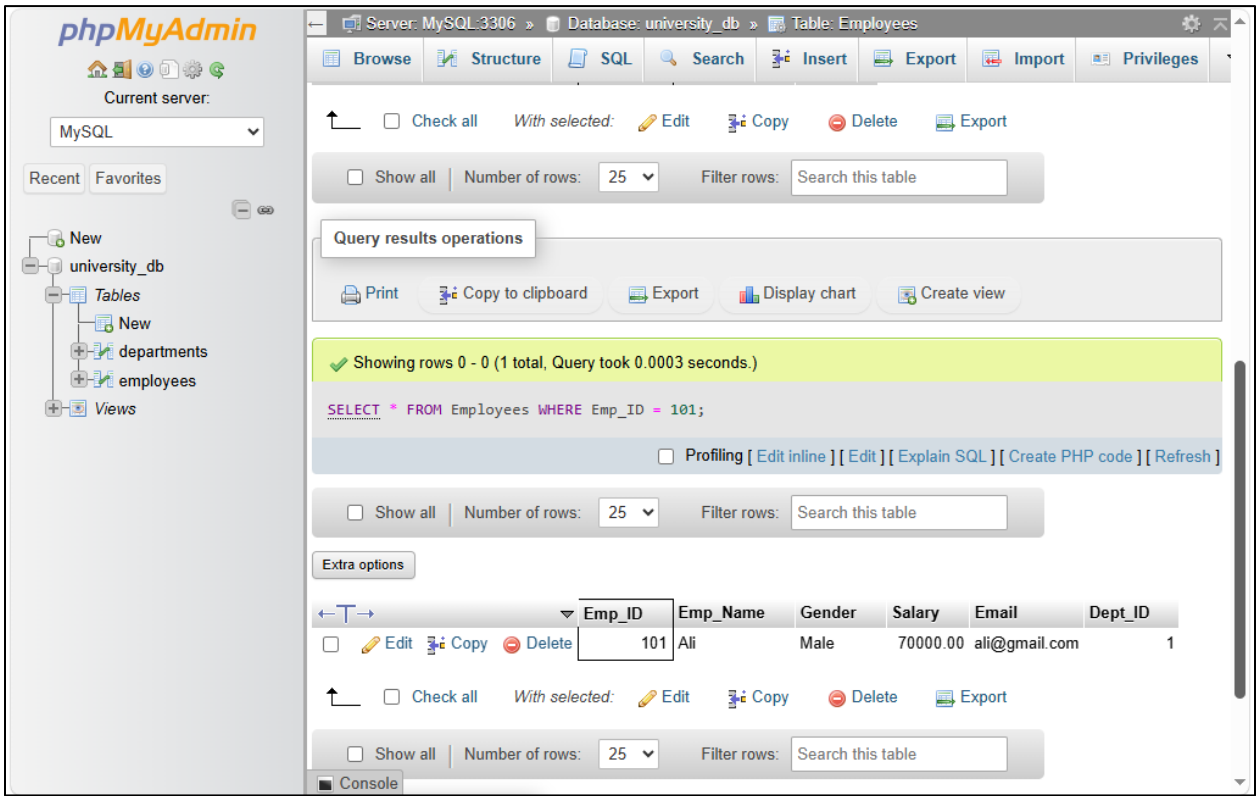
SELECT *
FROM Employee_Salary_View
WHERE Emp_ID = 101;

SELECT *
FROM Employees
```

WHERE Emp_ID = 101;



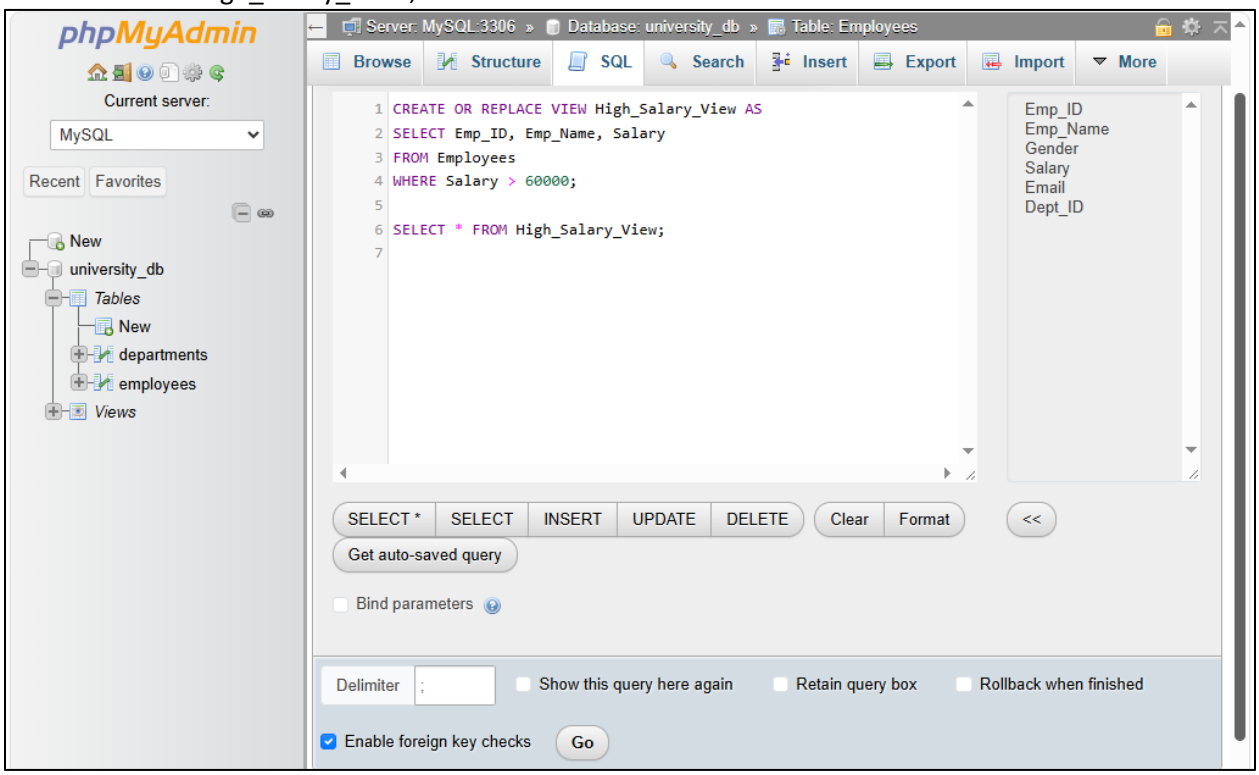


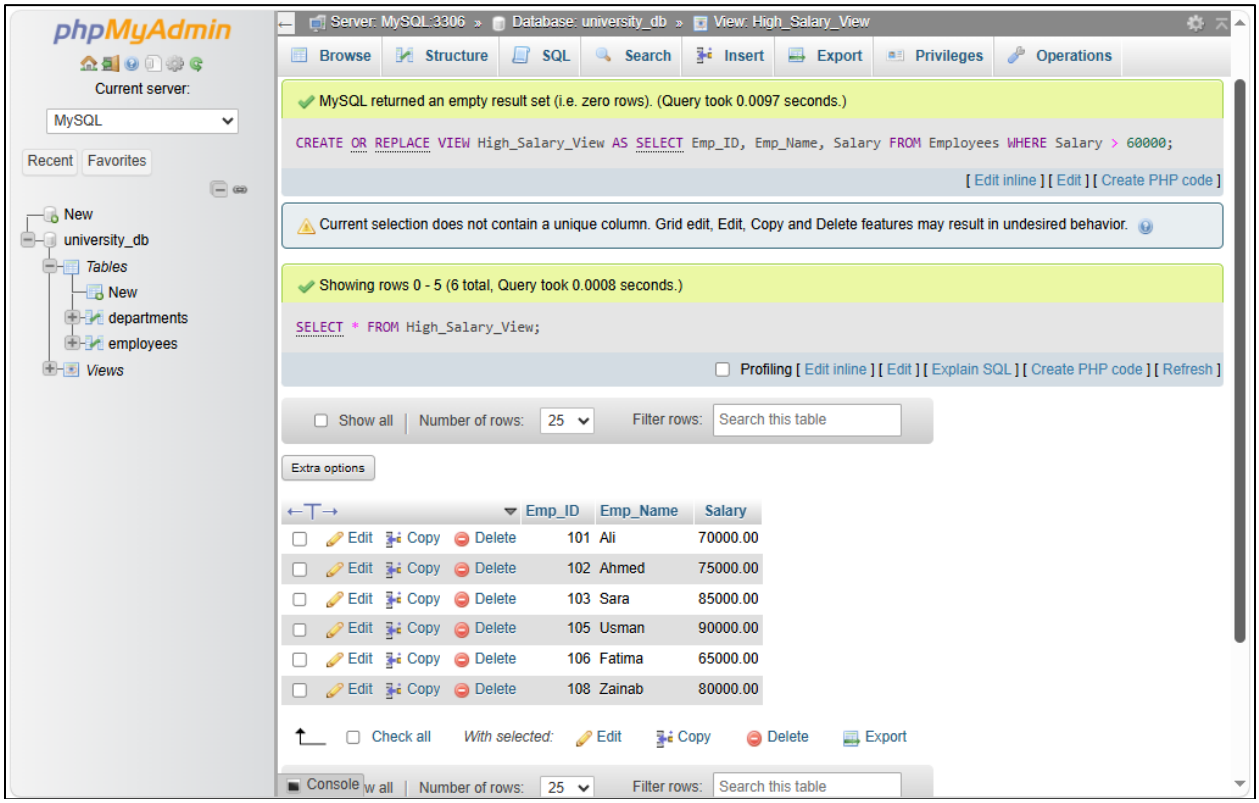


Task 9: Replace an existing view

```
CREATE OR REPLACE VIEW High_Salary_View AS
SELECT Emp_ID, Emp_Name, Salary
FROM Employees
WHERE Salary > 60000;
```

SELECT * FROM High_Salary_View;

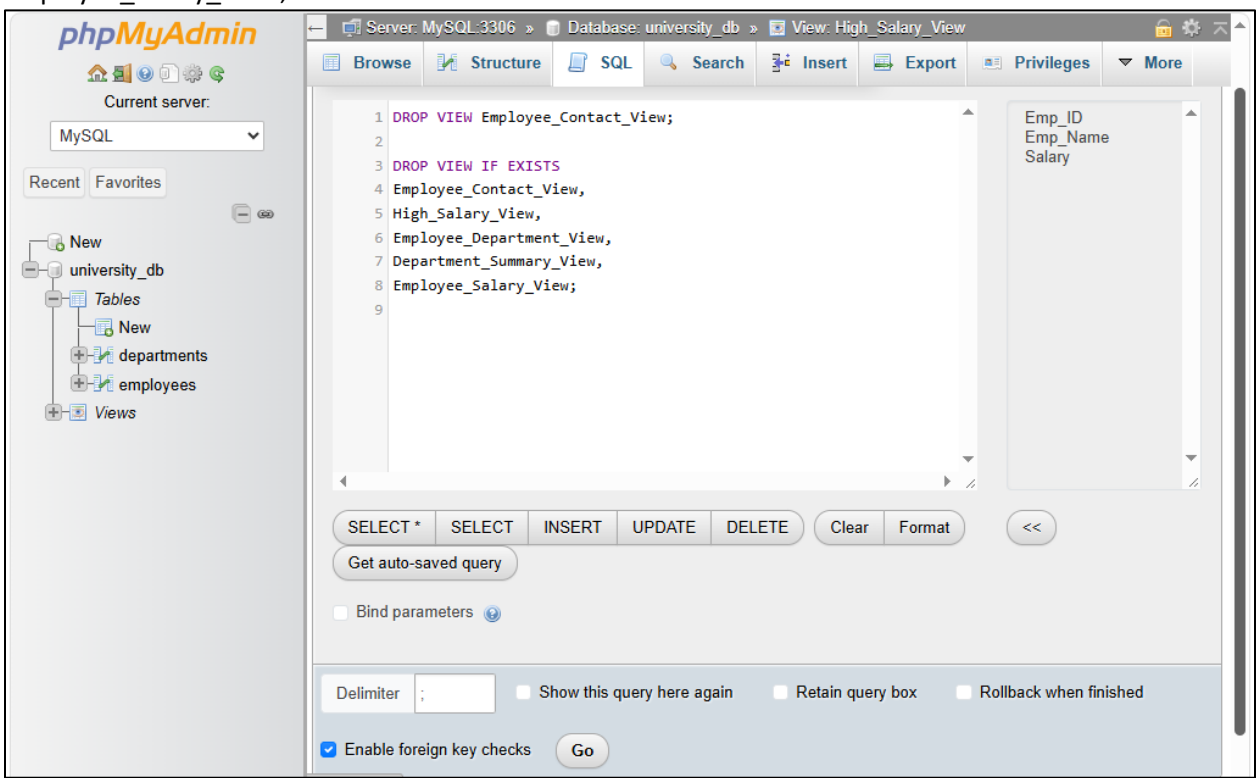


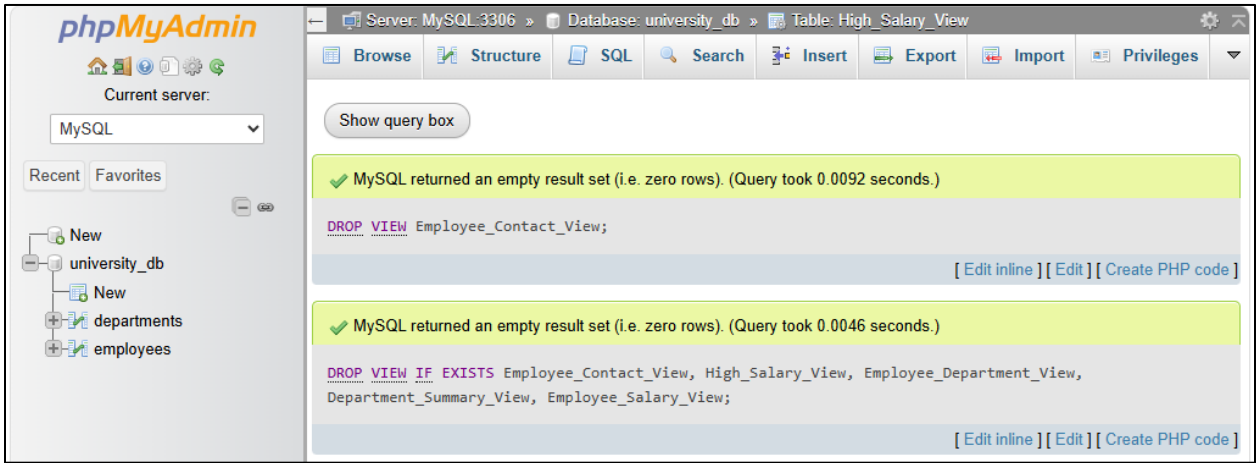


Task 10: Drop views

DROP VIEW Employee_Contact_View;

DROP VIEW IF EXISTS
Employee_Contact_View,
High_Salary_View,
Employee_Department_View,
Department_Summary_View,
Employee_Salary_View;





Conclusion

In this lab, a database with Departments and Employees tables was created and several types of views were built on it: a simple view, a filtered view, a join view, and an aggregate view. It was observed that a view stores no data of its own, and that updating a simple view (Employee_Salary_View) also changes the base Employees table. CREATE OR REPLACE VIEW was used to change the condition of an existing view without dropping it, and DROP VIEW and DROP VIEW IF EXISTS were used to remove views. Views therefore make queries shorter and reusable, and they help to restrict access to sensitive columns.